

DEVOPS MASTER COURSE

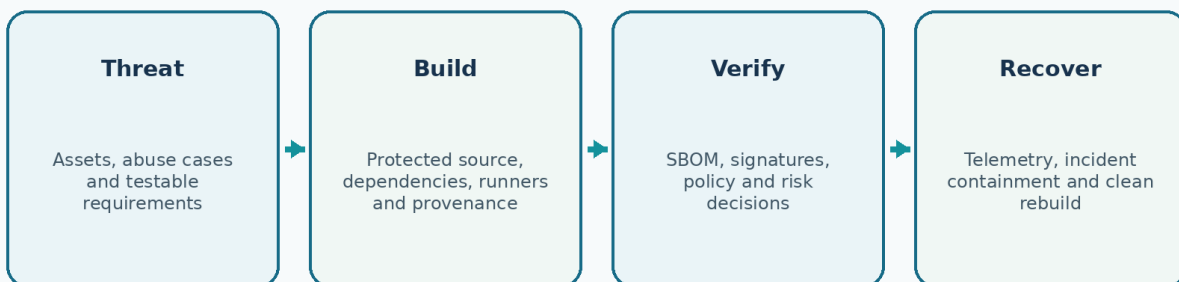
Volume 10E

DevSecOps & Software-Supply-Chain Security

Threat Modeling • Secure CI/CD • SAST/DAST • SBOM/VEX • SLSA • Sigstore •
Policy as Code • Incident Recovery

Chapters 41-55 | 2026 Edition

VOLUME 10E DELIVERY SYSTEM



Design → Build → Verify → Promote → Operate → Recover → Improve

Production objective: build fast without creating an untraceable path from source code to production by connecting threats, identities, dependencies, builds, artifacts, policy, evidence, runtime detection, governance, and recovery.

About This Part

Part E treats DevSecOps as a production operating model rather than a scanner stage. It protects repository change, dependencies, build infrastructure, runner identity, artifacts, Kubernetes and cloud configuration, SBOM and VEX, provenance, signatures, promotion, policy, vulnerability decisions, evidence, incident response, and clean-room recovery as one trust chain. This regenerated edition preserves the established Chapters 41–55 scope while aligning the roadmap with completed Parts D and F.

Graduation principle: a secure release claim is credible only when an independent reviewer can reconstruct what source, dependencies, tools, identities, policy, artifact, deployment decision, runtime evidence, exception, and recovery procedure produced it.

Volume 10 Roadmap

Part	Coverage	Status
10A	CI/CD foundations and pipeline engineering	Complete
10B	Jenkins administration and enterprise engineering	Complete
10C	GitHub Actions and GitLab CI/CD enterprise engineering	Complete
10D	Kubernetes delivery, GitOps and progressive delivery	Complete
10E	DevSecOps and software-supply-chain security	This book
10F	Enterprise projects, troubleshooting and final capstone	Complete

Enterprise Lab Safety and Evidence Standard

- Use synthetic secrets, disposable repositories, restricted cloud roles, nonproduction signing identities, and controlled vulnerable fixtures.
- Do not publish tokens, OIDC assertions, private keys, internal vulnerability details, exploit payloads, sensitive architecture, or unredacted scanner output.
- Pin tools, actions, images, policies, and dependencies to reviewed versions or immutable digests and record resolved identities.
- Separate untrusted validation from trusted release, signing, promotion, and production deployment execution.
- Define stop conditions, rollback, isolation ownership, evidence preservation, and cleanup before compromised-runner, malicious-artifact, policy-outage, or recovery exercises.

How to Use This Book

1. Start each chapter by identifying the threat, asset, trust boundary, attacker goal, business impact, and recovery requirement.
2. Implement controls at the point where context is strongest: developer feedback, pull request, protected build, registry, promotion, deployment admission, and runtime.
3. Treat scanner output as evidence requiring ownership, context, disposition, and retention rather than as an automatic release decision.
4. Exercise missing, invalid, stale, revoked, unavailable, bypassed, and compromised states deliberately.

5. Retain a sanitized evidence chain linking requirement, source, tool version, artifact digest, decision, deployment, finding, exception, and recovery.

Part Learning Outcomes

- Translate threats, business impact, and obligations into owned, testable delivery requirements.
- Protect repositories, dependencies, runner fleets, identities, secrets, artifacts, and deployment paths as one trust chain.
- Build high-signal SAST, SCA, secret, DAST, API, fuzzing, container, Kubernetes, IaC, and cloud configuration controls.
- Generate, validate, retain, and operationalize SBOM, VEX, provenance, attestations, and signatures.
- Operate policy as code, vulnerability triage, exceptions, security observability, evidence, and governance.
- Contain supply-chain compromise and restore trusted delivery through clean-room recovery.

Table of Contents

Chapter 41	DevSecOps Operating Model, Threat Modeling and Security Requirements
Chapter 42	Source Repository Security, Change Governance and Secret Prevention
Chapter 43	Dependency Governance, SCA and License Risk
Chapter 44	Secure Build Systems, Runner Fleets, Identity and Isolation
Chapter 45	SAST, Secret Scanning, Code Review and Developer Feedback
Chapter 46	DAST, API Security, Fuzzing and Runtime Validation
Chapter 47	Container, Kubernetes, IaC and Cloud Configuration Security
Chapter 48	SBOM, VEX, Component Identity and Vulnerability Intelligence
Chapter 49	SLSA, in-toto, Provenance and Reproducible Builds
Chapter 50	Sigstore, Cosign Signing, Verification and Artifact Promotion
Chapter 51	Policy as Code, Kubernetes Admission and Deployment Authorization
Chapter 52	Vulnerability Triage, Risk Scoring, SLAs and Exceptions
Chapter 53	Security Observability, Evidence Pipelines, Metrics and Governance
Chapter 54	Supply-Chain Incident Response and Clean-Room Recovery
Chapter 55	Enterprise DevSecOps Platform, Governance and Final Security Capstone
Part E Security Command and Evidence Reference	
Production Security Graduation Checklist	
Ten Additional DevSecOps Portfolio Projects	
90-Day DevSecOps Practice Plan	
Glossary	
Official Documentation Index	

Chapter 41: DevSecOps Operating Model, Threat Modeling and Security Requirements

Turning business risk and attacker behavior into owned, testable delivery controls.

Why this chapter matters: Adding scanners without ownership, requirements, decision authority, or recovery creates noise and false assurance.

Learning Objectives

- Explain shared accountability in a production delivery system.
- Design threat modeling with explicit trust boundaries.
- Operate security requirements using measurable evidence.
- Troubleshoot control placement and champions and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Shared accountability

Product teams own secure implementation and remediation; platform teams own paved roads; security defines and validates controls; operations protects runtime; risk owners accept residual risk. Shared responsibility must never mean unclear responsibility.

Threat modeling

Model assets, actors, trust boundaries, identities, data flows, abuse cases, attacker goals, likelihood, business impact, detectability, and recovery. Revisit when exposure, architecture, dependencies, or incidents change.

Security requirements

Translate threats into verifiable statements such as protected source, approved dependency origin, isolated build, immutable artifact, provenance, least privilege, admission policy, or maximum remediation time.

Control placement and champions

Place controls where context is strongest and feedback is useful. Security champions improve local decisions and education but do not replace specialists or accountable risk authority.

Architecture and Operating Decisions

- Define security outcomes before selecting products.
- Link every rule to a threat, requirement, obligation, or control objective.
- Assign owner, evidence, failure behavior, escalation, and exception authority.
- Measure signal quality, escape rate, decision latency, remediation time, and recovery.
- Treat exceptions as production objects with scope, compensating controls, expiry, and review.

Reference Configuration or Command Pattern

```
security_contract:
  source: {reviewed_change: true, protected_release: true}
  build: {isolated: true, dependency_lock: required, provenance: required}
  artifact: {digest: required, sbom: required, signature: required}
  deploy: {workload_identity: required, policy_bundle: prod-v12}
  operate: {evidence: centralized, rollback_tested: true}
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.

- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Teams bypass controls	Rules are slow, noisy or opaque	Reduce false positives and provide supported remediation	Bypass cases and decision latency
Threat model is stale	No owner or review trigger	Attach review to architecture, dependency, exposure and incident changes	Model version and review record
Security owns every finding	Service ownership and risk authority are unclear	Map services to owners and define escalation	Ownership map and decisions
Controls pass but incidents repeat	Failed assumptions are not connected to control redesign	Map incident causes to negative tests and remediation	Post-incident validation

Hands-on Lab 41: Create a Threat-Driven Security Delivery Contract

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Map assets, identities, trust boundaries, privileged actions, data, artifacts, targets, and evidence stores.
2. Document abuse cases for source, dependency, build, artifact, deployment, runtime, and recovery.
3. Prioritize threats by impact and uncertainty.
4. Convert high-priority threats into testable requirements.
5. Assign each requirement to control location, owner, evidence, failure behavior, and exception authority.
6. Run a tabletop review across product, platform, security, operations, and business risk.
7. Publish the contract and review triggers.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. What distinguishes DevSecOps from shift left?

Shift left moves feedback earlier; DevSecOps also integrates ownership, platform design, deployment, runtime, evidence, risk decisions, and recovery.

2. Who accepts residual risk?

An accountable authority with sufficient business and technical context, informed by security and service owners.

3. Why can more scanners reduce security?

Duplicate and low-signal findings create fatigue, inconsistency, delay, and incentives to bypass controls.

4. What makes a security requirement useful?

It is threat-linked, testable, owned, measurable, and has defined failure and exception behavior.

Chapter Summary

- DevSecOps is an operating model, not a stage.
- Threats become testable requirements and owned controls.
- Champions accelerate decisions but do not replace accountability.
- Security effectiveness includes evidence and recovery.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 42: Source Repository Security, Change Governance and Secret Prevention

Protecting the first high-trust boundary in the software supply chain.

Why this chapter matters: Weak repository identity, administration, workflow ownership, release references, or secret hygiene can invalidate every downstream control.

Learning Objectives

- Explain identity and access in a production delivery system.
- Design protected change with explicit trust boundaries.
- Operate sensitive paths using measurable evidence.
- Troubleshoot secret prevention and response and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Identity and access

Use managed account lifecycle, strong authentication, least privilege, periodic access review, separate human and workload identities, and rapid removal of dormant or compromised access.

Protected change

Require reviewed pull or merge requests, checks for the exact commit, restricted force push, sensitive-path ownership, and protected release tags. Emergency change needs equivalent evidence and reconciliation.

Sensitive paths

Apply stronger ownership to workflows, build images, dependency locks, infrastructure code, policy bundles, signing configuration, CODEOWNERS, and deployment manifests.

Secret prevention and response

Combine local checks, push protection, server scanning, provider detection, education, and canaries. Revoke first, investigate use, rotate dependents, search logs and artifacts, then consider history rewriting.

Architecture and Operating Decisions

- Separate repository administration from ordinary contribution.
- Require current-commit checks and re-review after material change.
- Protect bot tokens, apps, webhooks, deploy keys, and workflow permissions.
- Treat signed commits or tags as identity evidence, not correctness proof.
- Assume clones, caches, logs, packages, and forks may retain exposed secrets.

Reference Configuration or Command Pattern

```
# sensitive path ownership example
.github/workflows/  @platform-security
/ci/                @platform-security
/policy/            @security-engineering
/dependencies.lock  @service-owner @security-engineering
/deploy/            @service-owner @platform-operations
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.

- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Secret scan is clean but token is active	Credential leaked outside scanned source or was suppressed	Revoke, investigate audit use and rotate dependents	Provider revocation and timeline
Check passed on old commit	Branch changed after approval or check	Require checks and review for current head SHA	Merge evidence for exact commit
Bot modifies protected workflow	App permissions are excessive	Narrow installation, token scope and path ownership	Negative authorization test
History rewrite considered complete	Existing clones and caches retain data	Treat credential as compromised permanently and monitor use	Rotation and investigation record

Hands-on Lab 42: Build a Secure Repository Baseline and Secret Drill

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Create managed roles for administrator, maintainer, contributor, reviewer, bot, and external contributor.
2. Configure protected branches, release tags, required current-commit checks, and sensitive-path ownership.
3. Restrict workflow and token permissions.
4. Run secret prevention locally and server-side using a synthetic credential.
5. Simulate an accepted secret and execute revoke, investigation, rotation, history, and notification steps.
6. Review bots, apps, deploy keys, webhooks, dormant users, and bypass authority.
7. Publish an access-review and emergency-change runbook.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. Why protect workflow files more strongly?

They can change build, credentials, artifact, signing, and deployment behavior without changing application code.

2. What should happen first after secret exposure?

Revoke or disable the credential before attempting cleanup.

3. Do signed commits prove trust?

They help attribute identity and integrity but do not prove review, authorization, or safe content.

4. Why require checks for the exact commit?

A later change can invalidate the evidence from an earlier tested revision.

Chapter Summary

- Repository governance begins the trust chain.
- Sensitive paths deserve stronger ownership.
- Secret response starts with revocation.
- Bots and applications are privileged identities.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 43: Dependency Governance, SCA and License Risk

Controlling external code, transitive relationships, update flow, and legal obligations.

Why this chapter matters: Dependencies expand attack surface and operational risk; unpinned, abandoned, compromised, or incompatible components can silently enter trusted builds.

Learning Objectives

- Explain inventory and resolution in a production delivery system.
- Design source and integrity with explicit trust boundaries.
- Operate sca and contextual risk using measurable evidence.
- Troubleshoot license and lifecycle and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Inventory and resolution

Use ecosystem lockfiles, deterministic resolution, package-manager metadata, registries, and SBOMs to know exact direct and transitive components. A manifest without the resolved graph is incomplete.

Source and integrity

Prefer approved registries, namespace controls, checksums, signatures or provenance, dependency confusion defenses, immutable versions, and private package boundaries.

SCA and contextual risk

Combine known vulnerability data with reachability, exploitability, exposure, asset criticality, runtime presence, compensating controls, and upgrade feasibility. Severity alone is not a complete decision.

License and lifecycle

Track license obligations, prohibited combinations, notices, source distribution requirements, maintenance health, end-of-life, and owner. Dependency removal is often the safest remediation.

Architecture and Operating Decisions

- Fail unknown package origins in trusted builds.
- Review lockfile changes and automated update identity.
- Separate developer convenience proxies from trusted release sources.
- Set risk-based remediation and exception SLAs.
- Continuously re-evaluate released artifacts when intelligence changes.

Reference Configuration or Command Pattern

```
dependency_policy:
  allowed_sources: [internal-proxy]
  lockfile_required: true
  integrity_required: true
  prohibited_licenses: [<policy-list>]
  critical_exposure_sla_hours: 24
  exception: {owner: required, expiry: required, compensating_controls: required}
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.

- Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Package name resolves to public registry	Dependency confusion or source order	Block public fallback and reserve internal namespaces	Resolution trace and denied test
SCA creates thousands of findings	No asset, reachability or duplicate context	Normalize and prioritize with runtime and ownership data	Reduced actionable queue
Update breaks build	Transitive or API compatibility changed	Use staged updates, tests, changelog and rollback	Dependency diff and test evidence
License risk discovered late	License policy absent from intake	Evaluate at resolution and inventory released artifacts	Decision and notice evidence

Hands-on Lab 43: Create a Governed Dependency Intake Pipeline

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Select a sample service with direct and transitive dependencies.
2. Generate and compare manifest, lockfile, resolved graph, and SBOM.
3. Route resolution through an approved proxy and deny an unauthorized origin.
4. Run SCA and enrich one finding with reachability, runtime, exposure, owner, and remediation.
5. Create an automated update pull request with tests and review ownership.
6. Evaluate licenses and produce required notice evidence.
7. Simulate a newly disclosed critical issue in a released artifact and execute reassessment.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. Why is a lockfile important?

It records the resolved dependency graph so builds can reproduce and review exact versions rather than re-resolve silently.

2. What is dependency confusion?

A package manager selects an attacker-controlled public package instead of the intended private dependency.

3. Why not gate only on severity?

Severity omits reachability, exposure, exploitability, asset value, compensating controls, and remediation risk.

4. What proves dependency governance works?

Approved resolution, exact inventory, contextual decisions, controlled updates, license evidence, and post-release monitoring.

Chapter Summary

- Resolved identity matters more than manifest intent.
- Source integrity and namespace control prevent substitution.
- SCA requires context and ownership.
- Dependencies have vulnerability, license, and lifecycle risk.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 44: Secure Build Systems, Runner Fleets, Identity and Isolation

Preventing untrusted source from inheriting production authority.

Why this chapter matters: Build infrastructure executes attacker-influenced code and often has network, registry, secret, signing, or deployment access; shared persistence can turn one job into cross-project compromise.

Learning Objectives

- Explain trust tiers in a production delivery system.
- Design ephemeral execution with explicit trust boundaries.
- Operate workload identity using measurable evidence.
- Troubleshoot build integrity and evidence and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Trust tiers

Separate untrusted pull-request validation, trusted branch builds, release builds, signing, and deployment. Each tier receives only the identity, network, secret, and artifact access required.

Ephemeral execution

Prefer fresh, immutable runners or agents with pinned images, no cross-job workspace, controlled cache, restricted egress, resource limits, and verified teardown.

Workload identity

Use short-lived OIDC or platform-issued identities with audience, subject, repository, branch, environment, workflow, and approval claims instead of long-lived cloud keys.

Build integrity and evidence

Record source, build definition, toolchain, dependencies, runner image, identity, outputs, logs, and provenance. Protect the control plane, worker images, caches, registries, and logs as supply-chain assets.

Architecture and Operating Decisions

- Never expose release or production credentials to forked or untrusted code.
- Keep signing independent from general build execution.
- Pin third-party actions, plugins, images, and toolchains.
- Restrict metadata services, container socket, host mounts, and privileged execution.
- Test runner compromise, cache poisoning, credential misuse, and clean rebuild.

Reference Configuration or Command Pattern

```
trust_tiers:
  pr-validation: {secrets: none, network: restricted, runner: ephemeral}
  protected-build: {identity: short-lived, registry: write-staging}
  release: {approval: required, provenance: required}
  signing: {artifact_input: digest-only, key_access: isolated}
  production-deploy: {environment: protected, identity: scoped}
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.

- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Fork job reads secret	Trust-tier or event condition is wrong	Stop job, revoke, inspect logs and isolate untrusted workflows	No-secret negative test
Runner persists malware	Worker or cache is reused without sanitization	Quarantine fleet, rebuild image and rotate identities	Fresh worker attestation
OIDC role is too broad	Subject/audience policy lacks constraints	Bind trust to repository, workflow, ref and environment	Denied claim variants
Build cannot be reproduced	Mutable toolchain or dependency input	Pin inputs and record build definition/provenance	Rebuild comparison

Hands-on Lab 44: Design and Attack-Test an Isolated Runner Fleet

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Map event types, code trust, identities, secrets, networks, caches, artifacts, and target permissions.
2. Create separate untrusted, protected, release, signing, and deployment execution tiers.
3. Configure short-lived workload identity with strict claims.
4. Run an untrusted fixture attempting secret, metadata, network, cache, and host access.
5. Build an artifact on an ephemeral worker and record runner/toolchain evidence.
6. Simulate worker compromise and execute quarantine, revocation, image rebuild, and clean release.
7. Publish fleet SLOs, patching, capacity, and forensic retention.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. Why are CI runners high-risk?

They execute attacker-influenced code while connected to trusted systems and can persist across jobs if isolation is weak.

2. What should an OIDC trust policy constrain?

Issuer, audience, repository or project, workflow or job, branch/tag, environment, and other stable authorization claims.

3. Why isolate signing?

A compromised build should not automatically gain authority to sign or promote arbitrary bytes.

4. What is a clean runner rebuild?

Recreation from reviewed images and configuration after revoking identities and excluding potentially compromised caches or artifacts.

Chapter Summary

- Build systems are privileged supply-chain infrastructure.
- Trust tiers separate untrusted code from release authority.
- Ephemeral workers and short-lived identity reduce persistence.
- Recovery requires quarantine, revocation, and clean rebuild.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 45: SAST, Secret Scanning, Code Review and Developer Feedback

Finding actionable defects early without turning security into a blocking-noise factory.

Why this chapter matters: Early feedback is valuable only when rules, ownership, baseline, suppression, triage, and remediation are trustworthy and understandable.

Learning Objectives

- Explain sast design in a production delivery system.
- Design secret scanning with explicit trust boundaries.
- Operate review quality using measurable evidence.
- Troubleshoot finding lifecycle and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

SAST design

Use language-aware analyzers, secure coding rules, data flow, custom rules for organizational patterns, and a risk-based set of blocking controls. Run fast checks locally or in pull requests and deeper analysis on trusted branches.

Secret scanning

Combine pattern, entropy, provider validation where safe, custom tokens, push protection, history scanning, artifact scanning, and revocation workflow. Masking output does not undo exposure.

Review quality

Security-relevant code review focuses on trust boundaries, authorization, data validation, cryptography, logging, error handling, concurrency, dependencies, infrastructure, and recovery, not only style.

Finding lifecycle

Normalize identity, deduplicate, assign service owner, link source revision, track state and SLA, support evidence-backed suppression, and measure false-positive, reopen, escape, and remediation rates.

Architecture and Operating Decisions

- Define a reviewed blocking-rule set with rule owners.
- Use baselines for legacy code without hiding new defects.
- Require suppressions to explain false-positive or accepted-risk rationale and expiry.
- Protect scanner configuration and custom rules as sensitive paths.
- Keep developer feedback specific, reproducible, and linked to secure remediation guidance.

Reference Configuration or Command Pattern

```
finding:
  rule: authz-object-scope
  source_revision: <sha>
  location: src/orders.py:118
  owner: orders-team
  status: confirmed
  risk_context: {internet_exposed: true, tenant_boundary: true}
  sla: 7d
  suppression: null
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Pull requests are blocked by noise	Broad rules lack tuning or baseline	Create high-signal blocking set and triage rule backlog	False-positive and override metrics
Secret alert is ignored	Ownership and revocation path are unclear	Route by credential provider and service owner	Revocation time
Finding disappears after code moves	Identity is tied only to line number	Use fingerprints and source revision lineage	Finding continuity
Suppression never expires	No owner or lifecycle control	Require expiry, review trigger and compensating control	Suppression audit

Hands-on Lab 45: Build a High-Signal Pull-Request Security Gate

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Select safe vulnerable fixtures for authorization, injection, secret, and insecure configuration.
2. Run fast SAST and secret checks on a pull request using pinned tools.
3. Classify rules as blocking, advisory, or deep-analysis.
4. Create finding fingerprints, owners, remediation links, and an evidence-backed suppression.
5. Measure latency, duplicates, false positives, and developer action time.
6. Fix the defects and prove the exact commit passes.
7. Write rule-change, emergency bypass, and revocation procedures.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. What makes a good blocking rule?

High confidence, meaningful impact, clear ownership, reproducible evidence, and a practical remediation path.

2. Why baseline legacy findings?

To prevent existing debt from hiding new defects while preserving an owned remediation backlog.

3. Is log masking enough for secrets?

No. The credential may already have been used, copied, cached, or retained elsewhere; revoke and investigate.

4. What should a suppression contain?

Finding identity, rationale, evidence, scope, owner, approver, compensating controls, expiry, and review trigger.

Chapter Summary

- Early feedback must be high-signal.
- Findings need stable identity and lifecycle.
- Secret detection is coupled to revocation.
- Review and scanner rules are security controls.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 46: DAST, API Security, Fuzzing and Runtime Validation

Testing deployed behavior, contracts, inputs, authorization, and failure handling.

Why this chapter matters: Static analysis cannot observe all routing, identity, configuration, state, protocol, or runtime interactions; dynamic tests add evidence but can damage systems if uncontrolled.

Learning Objectives

- Explain dast scope in a production delivery system.
- Design api security with explicit trust boundaries.
- Operate fuzzing using measurable evidence.
- Troubleshoot runtime validation and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

DAST scope

Test an authorized nonproduction target with known routes, authentication, seed data, rate limits, stop conditions, and rollback. Distinguish passive observation, active scanning, authenticated testing, and destructive payloads.

API security

Validate object and function authorization, authentication, schema, mass assignment, rate limits, pagination, idempotency, replay, tenant isolation, error leakage, and inventory of versions and endpoints.

Fuzzing

Generate malformed or unexpected inputs to find crashes, hangs, memory errors, parser bugs, and invariant violations. Use reproducible seeds, corpus management, coverage, resource limits, and minimization.

Runtime validation

Combine synthetic transactions, security assertions, header and TLS checks, canary verification, WAF behavior, logs, traces, and side-effect checks. A scanner success status is not proof of user-path safety.

Architecture and Operating Decisions

- Obtain authorization and define target, payload, rate, window, data, stop condition, and cleanup.
- Use synthetic tenants and accounts to test authorization boundaries.
- Run dangerous tests only in isolated environments.
- Retain request/response evidence without secrets or personal data.
- Connect dynamic findings to source, owner, deployment, and remediation validation.

Reference Configuration or Command Pattern

```
dynamic_test_plan:
  target: https://staging.example.test
  authorization: change-1234
  identities: [tenant-a-user, tenant-b-user, admin]
  max_rps: 5
  destructive_tests: false
  stop_conditions: [error_rate_gt_5pct, data_integrity_alarm]
  cleanup: reset-synthetic-data
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.

- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Scanner causes instability	Rate, payload or environment limits were undefined	Stop test, restore service and narrow plan	Recovery and revised safeguards
API test misses authorization flaw	Only happy-path identity was used	Add cross-tenant and role matrix tests	Denied access evidence
Fuzzing finds unreproducible crash	Seed and environment were not retained	Persist minimal input, version and resource context	Deterministic reproduction
DAST result is false positive	Proxy, WAF or test data changes behavior	Reproduce manually and correlate server evidence	Validated disposition

Hands-on Lab 46: Run an Authorized API Security Qualification

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Create a nonproduction API with synthetic tenants, roles, objects, logs, traces, and reset script.
2. Document authorization, rate, payload, stop, and cleanup controls.
3. Run schema and negative tests for authentication, object authorization, mass assignment, idempotency, and rate limits.
4. Run safe authenticated DAST and correlate requests with server telemetry.
5. Fuzz one parser or endpoint with bounded resources and retained seeds.
6. Fix one issue and execute exact remediation validation.
7. Publish a sanitized qualification report and residual risk.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. Why is authenticated DAST important?

Many routes, business functions, and authorization flaws are invisible to unauthenticated scans.

2. What is BOLA?

Broken object-level authorization: a caller can access another object by changing its identifier without proper authorization.

3. What makes fuzzing evidence useful?

A minimized reproducible input, exact build, environment, stack trace, impact, owner, and regression test.

4. Why define stop conditions?

Dynamic tests can consume capacity, corrupt data, trigger defenses, or harm shared environments.

Chapter Summary

- Dynamic testing observes deployed behavior.
- API security requires identity and tenant matrices.
- Fuzzing depends on reproducibility and containment.
- Authorization and cleanup are part of the test design.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 47: Container, Kubernetes, IaC and Cloud Configuration Security

Preventing insecure deployment intent from becoming trusted runtime state.

Why this chapter matters: Secure application code can still be exposed by vulnerable images, excessive privileges, public networking, weak identity, unsafe defaults, or cloud and Kubernetes misconfiguration.

Learning Objectives

- Explain container image security in a production delivery system.
- Design kubernetes security with explicit trust boundaries.
- Operate iac security using measurable evidence.
- Troubleshoot cloud configuration and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Container image security

Use minimal reviewed bases, digest pinning, non-root execution, read-only filesystems where practical, dropped capabilities, no embedded secrets, patch policy, malware scanning, and provenance. Scanner results need runtime context.

Kubernetes security

Apply namespace and RBAC boundaries, workload identity, Pod Security standards, NetworkPolicy, resource limits, seccomp, admission, secret handling, audit, and controlled cluster administration.

IaC security

Analyze rendered plans and manifests for public exposure, encryption, logging, backup, identity, data retention, and organizational policy. Evaluate actual change and target context, not source snippets alone.

Cloud configuration

Use landing-zone guardrails, private connectivity, centralized audit, policy, asset inventory, privileged access, key management, and continuous posture monitoring. Define ownership for managed-service defaults and drift.

Architecture and Operating Decisions

- Scan both source and resolved/rendered deployment artifacts.
- Treat base images, modules, charts, actions, and policies as dependencies.
- Block critical privilege and exposure patterns with tested policy.
- Separate build-time findings from runtime presence and reachability.
- Verify remediation in the deployed target and retain before/after evidence.

Reference Configuration or Command Pattern

```
securityContext:
  runAsNonRoot: true
  seccompProfile: {type: RuntimeDefault}
  capabilities: {drop: [ALL]}
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
# Pair with resource limits, workload identity, NetworkPolicy and admission tests.
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.

- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Image passes but pod is privileged	Runtime manifest overrides image assumptions	Evaluate image and deployment together	Admission denial and runtime spec
IaC scan misses public resource	Dynamic value resolved only in plan	Scan reviewed plan and cloud policy state	Plan plus deployed asset query
Policy blocks system workload	Rule lacks scoped exceptions or platform tests	Add controlled namespace/identity exception with expiry	Positive/negative test suite
Fix changes source but not runtime	Old digest or failed reconciliation remains deployed	Verify runtime digest and configuration	Observed deployed state

Hands-on Lab 47: Qualify a Containerized Workload for Production

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Build a sample image from a minimal pinned base and generate its digest.
2. Scan packages, secrets, malware indicators, configuration, and SBOM using pinned tools.
3. Create hardened Kubernetes resources with workload identity, policies, limits, probes, and network controls.
4. Evaluate source manifests, rendered output, server dry-run, admission, and runtime state.
5. Introduce a privileged container, public service, broad role, and missing encryption fixture and confirm detection.
6. Remediate and verify the deployed workload.
7. Publish an evidence-backed risk disposition.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. Why scan rendered IaC or manifests?

Variables, modules, overlays, templates, defaults, and target context determine the actual resources.

2. Does a minimal image eliminate vulnerabilities?

No. It reduces surface area but still requires trusted origin, patching, inventory, configuration, and runtime controls.

3. Why use workload identity?

It provides scoped short-lived access without embedding long-lived cloud credentials in pods.

4. What is defense in depth here?

Secure image, restricted runtime, network isolation, identity, admission, cloud guardrails, telemetry, and recovery all cooperate.

Chapter Summary

- Application security includes deployment configuration.
- Rendered and runtime state matter.
- Images and reusable modules are dependencies.
- Remediation must be verified in the target.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 48: SBOM, VEX, Component Identity and Vulnerability Intelligence

Building accurate inventory and communicating product-specific vulnerability status.

Why this chapter matters: An SBOM is useful only when component identity, relationships, artifact linkage, generation method, freshness, distribution, and downstream decisions are trustworthy.

Learning Objectives

- Explain sbom content and identity in a production delivery system.
- Design generation and coverage with explicit trust boundaries.
- Operate vex using measurable evidence.
- Troubleshoot vulnerability intelligence and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

SBOM content and identity

Record components, versions, package URLs or CPE where applicable, hashes, suppliers, licenses, dependencies, and relationships. Link the SBOM to an immutable artifact digest and build provenance.

Generation and coverage

Combine source, build, package-manager, binary, image, and runtime perspectives as needed. Document scope, omissions, tool versions, evidence, and confidence.

VEX

A VEX statement communicates whether a product is affected, not affected, fixed, or under investigation, with justification and product/version identity. It is not a blanket dismissal of upstream vulnerabilities.

Vulnerability intelligence

Continuously correlate SBOMs with advisories, exploitability, EPSS or equivalent signals, reachability, runtime presence, asset criticality, exposure, compensating controls, and remediation availability.

Architecture and Operating Decisions

- Generate SBOMs during trusted builds and retain them with artifact identity.
- Validate schema and component completeness against lockfiles and image contents.
- Sign or attest SBOMs and VEX where the trust model requires it.
- Version VEX statements, evidence, owners, and expiry/review triggers.
- Notify owners when new intelligence affects already released artifacts.

Reference Configuration or Command Pattern

```
artifact: registry.example/payments@sha256:<digest>
sbom: {format: cyclonedx-json, digest: sha256:<sbom-digest>}
vulnerability: CVE-YYYY-NNNN
vex:
  status: not_affected
  justification: vulnerable_code_not_in_execute_path
  evidence: <reachability-report>
  owner: payments-security
  review_by: 2026-09-01
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.

- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
SBOM omits transitive package	Generator lacks ecosystem or binary coverage	Combine sources and validate against lockfile/image	Coverage comparison
Finding cannot map to artifact	SBOM is stored by filename only	Bind SBOM digest to immutable artifact digest	Registry/attestation link
VEX suppresses every product	Product identity or justification is too broad	Scope to exact product/version and evidence	Targeted reassessment
New CVE reaches owners late	No continuous correlation or ownership map	Index released SBOMs and route intelligence events	Notification and SLA metrics

Hands-on Lab 48: Create an SBOM-to-VEX Decision Pipeline

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Build a sample artifact with direct and transitive dependencies.
2. Generate SPDX and/or CycloneDX SBOM tied to the artifact digest.
3. Compare SBOM contents with lockfile, image packages, and source expectations.
4. Correlate one known-safe training vulnerability and enrich it with reachability and runtime context.
5. Create a product-scoped VEX statement with evidence and review date.
6. Sign or attest the SBOM/VEX relationship in the lab trust model.
7. Simulate new advisory intelligence and route reassessment to the owner.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. What is the most important SBOM link?

The immutable identity of the exact artifact or product version the inventory describes.

2. Does VEX mean a CVE is false?

No. It states the product-specific impact status and justification for a particular product/version.

3. Why can two SBOM tools disagree?

They inspect different evidence layers, ecosystems, binaries, files, and relationship models.

4. What makes vulnerability intelligence operational?

Continuous correlation, contextual risk, ownership, SLA, decision evidence, and re-evaluation of released artifacts.

Chapter Summary

- SBOM quality depends on identity and coverage.
- VEX communicates scoped product status.
- Intelligence changes after release.
- Inventory must drive owned decisions.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 49: SLSA, in-toto, Provenance and Reproducible Builds

Establishing verifiable evidence about how artifacts were produced.

Why this chapter matters: A source commit and artifact checksum do not reveal whether the expected build definition, isolated infrastructure, trusted identity, dependencies, or parameters produced the bytes.

Learning Objectives

- Explain slsa framework in a production delivery system.
- Design provenance with explicit trust boundaries.
- Operate in-toto attestations using measurable evidence.
- Troubleshoot reproducibility and hermeticity and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

SLSA framework

SLSA organizes artifact integrity controls across build tracks and levels. Apply it as a risk-improvement model rather than a badge or guarantee of vulnerability-free software.

Provenance

Provenance identifies subject artifacts, builder identity, build type, invocation, source materials, parameters, environment and metadata. Consumers must verify both signature and policy claims.

in-toto attestations

in-toto statement and predicate formats enable signed claims about steps or properties. The trust policy defines acceptable identities, builders, source, workflow, and artifact relationships.

Reproducibility and hermeticity

Hermetic builds control network and inputs; reproducible builds allow independent builds to produce equivalent outputs. Both improve confidence but can be difficult with timestamps, nondeterminism, external services, and platform variation.

Architecture and Operating Decisions

- Define the threat and target assurance before selecting a SLSA level.
- Use protected builders and short-lived identities for provenance generation.
- Verify provenance at promotion or deployment, not merely store it.
- Record all resolved materials and reject mutable source references.
- Investigate nonreproducibility rather than normalizing unexplained differences.

Reference Configuration or Command Pattern

```
verification_policy:
  subject_digest: required
  builder_identity: trusted-release-builder
  source_repository: https://git.example/payments
  source_revision: protected-tag
  build_type: org.example/container-build/v2
  parameters: allowlisted
  materials: immutable
  signature: verified
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.

- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Provenance exists but wrong builder accepted	Verifier trusts any signer or predicate	Constrain builder, source, build type and claims	Negative provenance tests
Materials contain branch name	Mutable source reference is recorded	Resolve and attest exact commit and dependency identities	Immutable materials
Rebuild differs unexpectedly	Timestamp, network, order or toolchain is nondeterministic	Capture environment and eliminate or explain variance	Diff and root-cause record
Attestation stored separately and lost	No artifact lifecycle binding	Attach in registry/evidence index and retain together	Retrieval by digest

Hands-on Lab 49: Generate and Verify Build Provenance

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Define a threat model and target SLSA assurance for a sample container build.
2. Pin source, dependency, builder image, and toolchain identities.
3. Build on a protected worker using short-lived identity.
4. Generate provenance describing the artifact digest, builder, build type, source and materials.
5. Verify valid provenance at promotion.
6. Create negative cases for wrong source, wrong builder, modified digest, mutable material, and invalid signature.
7. Attempt a second build and analyze reproducibility differences.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. What does SLSA protect?

Primarily artifact integrity and the path from source through build; it does not prove the software is vulnerability-free or correctly authorized for deployment.

2. Why verify provenance policy?

A valid signature from an unapproved builder or source is still unacceptable.

3. What is hermeticity?

The build obtains all inputs from declared controlled sources rather than uncontrolled network or host state.

4. What is reproducibility useful for?

Independent agreement increases confidence and reveals hidden nondeterminism or undeclared inputs.

Chapter Summary

- SLSA is a risk-improvement framework.
- Provenance must be verified against policy.
- Identity and immutable materials are central.
- Hermeticity and reproducibility expose hidden inputs.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 50: Sigstore, Cosign Signing, Verification and Artifact Promotion

Using identity, transparency, signatures and attestations to authorize immutable artifacts.

Why this chapter matters: Signing adds integrity and identity evidence, but weak identity policy, mutable tags, stolen keys, unverified claims, or bypassable promotion can create signed malware.

Learning Objectives

- Explain sigstore model in a production delivery system.
- Design cosign artifacts with explicit trust boundaries.
- Operate trust policy using measurable evidence.
- Troubleshoot promotion and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Sigstore model

Sigstore combines artifact signing, identity certificates or keys, transparency, and verification. Keyless signing uses workload or human identity; key-based signing requires protected custody and rotation.

Cosign artifacts

Cosign can sign container images and other blobs and attach attestations. Verification starts with the immutable artifact digest and then evaluates signer identity, issuer, claims, transparency, and predicate.

Trust policy

Define accepted issuers, subjects, repositories, workflows, branches/tags, environments, builders, predicate types, timestamps, revocation, and offline or availability behavior.

Promotion

Promote the same digest through registries or environment references. Require signature, provenance, SBOM, vulnerability and policy evidence before authorization; preserve the decision and reject tag substitution.

Architecture and Operating Decisions

- Sign only after protected build completion and before untrusted mutation.
- Separate signing authority from ordinary build credentials.
- Verify identity and claims, not only cryptographic validity.
- Plan certificate, transparency, key, registry, and verification-service outages.
- Revoke or distrust compromised identities and quarantine affected artifacts.

Reference Configuration or Command Pattern

```
# conceptual commands; use current pinned Cosign syntax
cosign sign <registry>/payments@sha256:<digest>
cosign attest --predicate provenance.json --type slsaprovenance <image>@sha256:<digest>
cosign verify --certificate-identity-regexp '<approved-workflow>' \
  --certificate-oidc-issuer '<approved-issuer>' <image>@sha256:<digest>
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.

- Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Signature verifies for attacker artifact	Trust policy accepts broad identity	Constrain issuer, subject, workflow, source and environment	Wrong-identity denial
Tag points to unsigned digest	Deployment verifies tag or not at all	Resolve and verify immutable digest	Runtime digest evidence
Signing service unavailable	No outage or queue policy	Pause release or use documented alternate authority	Outage exercise
Signing identity compromised	Artifacts remain trusted indefinitely	Revoke/distrust identity, inventory scope and quarantine	Affected artifact list and response

Hands-on Lab 50: Implement a Signed Artifact Promotion Gate

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Create a lab signing identity and explicit trust policy.
2. Build an image and record its immutable digest.
3. Generate SBOM and provenance attestations and sign the artifact.
4. Verify digest, issuer, identity, workflow/source claims, and predicates before promotion.
5. Create negative cases for altered digest, wrong signer, wrong source, missing attestation, and stale trust policy.
6. Promote the same digest to a production environment reference.
7. Simulate identity compromise and execute distrust, inventory, quarantine, and rebuild.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. What does a signature prove?

That a holder of an accepted signing identity authorized specific bytes; policy and provenance determine whether that identity and build context are acceptable.

2. Why verify by digest?

Tags are mutable and can be retargeted after signing or approval.

3. What is keyless signing?

Signing based on short-lived identity credentials and certificates rather than a long-lived private key managed by the user.

4. Can signed malware exist?

Yes. Signing is not code review or vulnerability analysis; a compromised or authorized builder can sign harmful bytes.

Chapter Summary

- Signing provides identity and integrity evidence.
- Verification policy is as important as signature creation.
- Promotion preserves immutable digest.
- Compromise requires distrust, quarantine and rebuild.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 51: Policy as Code, Kubernetes Admission and Deployment Authorization

Making deployment decisions consistent, testable, observable, and recoverable.

Why this chapter matters: Policy can prevent unsafe deployment only when rules map to requirements, inputs are trustworthy, decisions are testable, exceptions expire, and outage behavior is designed.

Learning Objectives

- Explain policy lifecycle in a production delivery system.
- Design evaluation points with explicit trust boundaries.
- Operate admission control using measurable evidence.
- Troubleshoot failure modes and exceptions and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Policy lifecycle

Define requirement, threat, rule, owner, tests, version, rollout, telemetry, exception, deprecation, and recovery. Treat policy bundles as release artifacts.

Evaluation points

Run local or pull-request checks for feedback, CI checks for rendered context, registry or promotion checks for artifact evidence, and admission checks for cluster truth. Avoid inconsistent duplicate logic.

Admission control

Built-in admission plus engines such as ValidatingAdmissionPolicy, Gatekeeper, Kyverno, or another approved tool can validate or mutate requests. Mutation can hide intent and must be bounded and observable.

Failure modes and exceptions

Choose fail-closed, fail-open, cached, or tiered behavior by risk and availability. Break-glass and waivers need scope, identity, evidence, compensating controls, expiry, monitoring, and reconciliation.

Architecture and Operating Decisions

- Link each policy to a documented requirement.
- Test positive, negative, boundary, exception, upgrade, timeout, and outage cases.
- Roll out in audit or warn mode before enforce where risk justifies it.
- Version policy and evidence with the release decision.
- Protect policy administration separately from application contribution.

Reference Configuration or Command Pattern

```
policy_release:
  bundle: prod-policy-v12
  requirements: [non-root, signed-image, approved-registry, resource-limits]
  mode: enforce
  outage_behavior: fail-closed-critical / cached-low-risk
  exceptions:
    required_fields: [scope, owner, approver, evidence, expiry]
  tests: [positive, negative, exception, timeout, upgrade]
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.

- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Valid workload is denied	Rule, data, API version or mutation differs	Inspect admission response and test fixture; roll back safely	Reproduced and corrected decision
Policy service outage blocks emergency fix	No tiered availability or break-glass	Use controlled emergency path and restore policy	Full decision timeline
Exception persists forever	Expiry not enforced	Automate expiration and alert owner before closure	Expired access denied
CI passes but admission fails	Rendered context or policy version differs	Use same tested bundle and server-side validation	Matched policy/version evidence

Hands-on Lab 51: Build and Operate a Deployment Authorization Policy

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Choose four requirements such as non-root, approved registry, signed image, resource limits, and restricted host access.
2. Implement rules in an approved policy engine and version the bundle.
3. Create positive, negative, boundary, exception, timeout, and API-version fixtures.
4. Run policy in audit mode and inspect false positives.
5. Enforce it for a sandbox namespace and verify admission telemetry.
6. Simulate policy-service outage and execute the documented behavior.
7. Create and expire one scoped exception automatically.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. Why map policy to requirements?

It provides rationale, ownership, review triggers, and evidence that the rule addresses a real risk.

2. When might fail-open be acceptable?

For lower-risk controls where availability impact exceeds immediate security risk and compensating detection and remediation exist.

3. Why is mutation risky?

It changes submitted intent, can hide ownership, complicate debugging, and may conflict with other controllers.

4. What makes a waiver safe?

Narrow scope, accountable owner and approver, evidence, compensating controls, expiry, monitoring, and closure validation.

Chapter Summary

- Policy is a governed product.
- Use layered evaluation without inconsistent logic.
- Admission needs explicit availability behavior.
- Exceptions are temporary, observable, and expiring.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 52: Vulnerability Triage, Risk Scoring, SLAs and Exceptions

Converting large finding streams into timely, defensible remediation decisions.

Why this chapter matters: Teams cannot remediate every finding immediately; simplistic severity gates create either paralysis or unsafe acceptance.

Learning Objectives

- Explain risk context in a production delivery system.
- Design triage workflow with explicit trust boundaries.
- Operate slas and prioritization using measurable evidence.
- Troubleshoot exceptions and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Risk context

Combine technical severity with exploitability, known exploitation, reachability, runtime presence, network exposure, privileges, data sensitivity, asset criticality, tenant impact, detection, compensating controls, and remediation complexity.

Triage workflow

Normalize and deduplicate findings, identify product and artifact, assign owner, reproduce, classify affected status, select remediation, set SLA, track validation, and retain decision evidence.

SLAs and prioritization

Use risk tiers and clocks that start from trustworthy awareness. Define pause, escalation, overdue, emergency, and exception behavior. Measure exposure window, not only ticket age.

Exceptions

Risk acceptance is not scanner suppression. It requires authority, business context, technical evidence, scope, compensating controls, monitoring, expiry, review trigger, and closure.

Architecture and Operating Decisions

- Use a transparent risk model and preserve inputs.
- Prioritize exploited and internet-exposed reachable weaknesses.
- Separate “not affected” technical determination from accepted residual risk.
- Reassess when intelligence, architecture, exposure, controls, or exploit availability changes.
- Measure backlog age, exposure time, reopen rate, exception debt, and SLA reliability by owner and service.

Reference Configuration or Command Pattern

```
risk_decision:
  finding: CVE-YYYY-NNNN
  artifact: sha256:<digest>
  affected: true
  context: {reachability: true, internet_exposed: true, privilege: low}
  action: upgrade
  sla: 24h
  compensating_controls: [temporary-route-block]
  owner: payments-team
  validation: required
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Critical backlog grows	Severity-only queue lacks ownership and capacity	Prioritize contextual exposure and assign service owners	Exposure-window reduction
Finding repeatedly reopens	Fix version not deployed or inventory is stale	Verify runtime artifact and reconciliation	Observed fixed digest
Risk accepted by security alone	Business authority and service context are missing	Route to accountable risk owner with evidence	Approved decision and expiry
Scanner suppression hides new scope	Suppression is broad or pattern-based	Bind to exact finding/product/version and review trigger	Scoped negative test

Hands-on Lab 52: Operate a Contextual Vulnerability Triage Board

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Import safe findings from dependency, container, IaC, and application scanners.
2. Normalize identity, remove duplicates, map artifacts to services and owners.
3. Enrich selected findings with reachability, exposure, runtime, asset, exploit, and control context.
4. Assign remediation SLAs using a transparent risk matrix.
5. Create one not-affected VEX decision and one temporary risk acceptance.
6. Remediate one issue and verify the deployed artifact.
7. Reassess after a simulated exploited-in-the-wild update.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. Why is CVSS insufficient by itself?

It describes technical severity but not product reachability, exposure, asset value, compensating controls, or exploit conditions.

2. What is the difference between not affected and accepted risk?

Not affected is a technical product-status determination; accepted risk acknowledges exposure but authorizes temporary residual risk.

3. What should an SLA measure?

The allowed exposure window for a risk tier, including escalation and validation, not merely ticket creation age.

4. Who owns remediation?

The service or product owner, supported by security and platform teams; risk acceptance belongs to an authorized business or engineering authority.

Chapter Summary

- Triage converts findings into decisions.
- Context changes priority.
- SLAs manage exposure windows.
- Exceptions require authority, evidence, expiry, and re-evaluation.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 53: Security Observability, Evidence Pipelines, Metrics and Governance

Making control health, release decisions, risk, and audit evidence continuously visible.

Why this chapter matters: A security program cannot improve or defend its claims when evidence is fragmented, mutable, sensitive, unactionable, or disconnected from services and releases.

Learning Objectives

- Explain security telemetry in a production delivery system.
- Design evidence pipeline with explicit trust boundaries.
- Operate metrics using measurable evidence.
- Troubleshoot governance and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Security telemetry

Collect repository, identity, runner, dependency, scanner, registry, signing, policy, deployment, runtime, vulnerability, exception, and incident signals with stable identities and timestamps.

Evidence pipeline

Index source revision, workflow, actor, runner, tool versions, artifact digest, SBOM, provenance, signature, policy, approval, deployment, findings, exceptions, and health. Protect integrity, access, retention, privacy, and legal constraints.

Metrics

Measure control coverage, signal precision, escape rate, decision latency, remediation exposure, exception debt, signed/provenanced release coverage, drift, unauthorized change, recovery readiness, and evidence completeness.

Governance

Use risk-based reviews, control ownership, platform/product responsibility, service catalogs, policy versioning, audit samples, exception councils, maturity roadmaps, and transparent improvement actions. Avoid ranking individuals by raw finding counts.

Architecture and Operating Decisions

- Create a canonical release and finding identity model.
- Preserve evidence immutably or with tamper-evident controls.
- Minimize secrets, personal data, source snippets, and sensitive architecture in telemetry.
- Link dashboards to owners and response runbooks.
- Sample evidence regularly by reconstructing releases and incidents.

Reference Configuration or Command Pattern

```
release_record:
  release_id: rel-2026-00123
  source_revision: <sha>
  build_identity: <workflow-subject>
  artifact_digest: sha256:<digest>
  sbom_digest: sha256:<digest>
  provenance: <uri>
  signature_policy: pass
  deployment_policy: pass
  environment: production
  health_verification: pass
```

exceptions: []

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Audit evidence cannot be found	No canonical index or retention owner	Create digest/revision-based evidence catalog	Independent reconstruction
Dashboard is green but controls fail	Metrics show tool uptime, not control outcome	Measure coverage, escape and sampled decision quality	Control test evidence
Logs expose secrets	Verbose output and broad access	Redact/minimize, rotate exposure and restrict retention	Privacy/security review
Teams game metric	Metric is tied to punishment or vanity targets	Use balanced outcome and quality measures	Behavioral and outcome review

Hands-on Lab 53: Build a Release Security Evidence Dashboard

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Define canonical identities for service, source, build, artifact, release, environment, finding, exception, and incident.
2. Collect representative evidence from repository through runtime.
3. Store sanitized evidence with integrity, access, retention, and ownership controls.
4. Create dashboards for coverage, failures, latency, findings, exceptions, and recovery readiness.
5. Reconstruct one release from the evidence index without using the original pipeline UI.
6. Introduce missing and conflicting evidence and alert the owner.
7. Run a governance review and prioritize three improvements.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. What is evidence completeness?

The percentage of required, valid, retrievable evidence present for the release or control decision.

2. Why not measure developers by vulnerability count?

Counts reflect service exposure, scanner coverage, codebase size, and noise and encourage hiding rather than improvement.

3. What makes evidence trustworthy?

Stable identity, integrity, access control, provenance, timestamp, retention, reproducibility, and independent verification.

4. What should a security dashboard enable?

An owner can understand control state, risk, required action, decision history, and recovery readiness.

Chapter Summary

- Observability covers control outcomes and evidence quality.
- Canonical identities connect the trust chain.
- Metrics should improve systems, not punish individuals.
- Governance samples real decisions and recovery.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 54: Supply-Chain Incident Response and Clean-Room Recovery

Containing compromised identities, builders, artifacts, policy, registries, or deployment paths.

Why this chapter matters: Ordinary application incident procedures are insufficient when the systems that produce and authorize releases may themselves be untrustworthy.

Learning Objectives

- Explain incident preparation in a production delivery system.
- Design containment with explicit trust boundaries.
- Operate clean-room rebuild using measurable evidence.
- Troubleshoot reintroduction and learning and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Incident preparation

Maintain asset and identity inventories, artifact and SBOM indexes, provenance, signing trust, runner images, logs, backups, revocation procedures, alternate communications, clean repositories, and recovery authority.

Containment

Stop unsafe builds and promotion, quarantine artifacts, revoke identities, disable compromised integrations, protect evidence, scope affected source/builders/artifacts/environments, and preserve business-critical service safely.

Clean-room rebuild

Use a separately trusted environment, reviewed source and dependencies, new identities, clean toolchains, isolated builders, regenerated artifacts, new provenance/signatures, and independent verification. Do not reuse suspect caches, workers, keys, or packages.

Reintroduction and learning

Restore in risk order, verify runtime digest and behavior, monitor for persistence, communicate impact, publish a blameless timeline, and convert root and contributing causes into owned tested improvements.

Architecture and Operating Decisions

- Pre-authorize emergency containment and revocation roles.
- Separate evidence preservation from destructive cleanup.
- Define criteria for declaring source, builder, registry, identity, and artifact trustworthy again.
- Inventory all releases that share the compromised component or identity.
- Rehearse clean-room build and deployment before an incident.

Reference Configuration or Command Pattern

```
clean_room_recovery:
  inputs: [reviewed-source-commit, approved-lockfiles, clean-builder-image]
  excluded: [old-cache, old-runner, compromised-key, mutable-tag]
  identities: newly-issued-short-lived
  outputs: [new-digest, new-sbom, new-provenance, new-signature]
  verification: [independent-review, policy, smoke, slo, threat-hunt]
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.

- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Builds continue after compromise	No global containment authority or dependency map	Disable trust paths, revoke identities and quarantine output	No new affected releases
Recovered artifact still suspect	Old cache, base or dependency reused	Rebuild from independently reviewed inputs	Clean-room provenance
Scope cannot be determined	Artifact and identity evidence is incomplete	Use registry, logs, SBOMs and deployment inventory; state uncertainty	Affected-set report
Service restored but trust not restored	Only runtime symptoms were addressed	Reissue identities, rebuild artifacts and verify production digest	Trust-restoration checklist

Hands-on Lab 54: Run a Compromised Runner and Malicious Artifact Game Day

Prerequisites

- A disposable repository, registry, CI system, and nonproduction target.
- Pinned security tools or known-safe equivalents.
- Synthetic data, secrets, findings, and signing identities.

Implementation

1. Define participants, authority, communications, evidence custody, blast radius, stop conditions, and synthetic indicators.
2. Simulate a runner credential misuse or altered build output.
3. Detect and declare the incident; stop release and promotion paths.
4. Revoke identities, quarantine artifacts, scope source/builds/environments, and preserve evidence.
5. Build the service in a clean room with new identity and protected inputs.
6. Generate new SBOM, provenance, signature, policy and deployment evidence.
7. Restore service, monitor, communicate, conduct post-incident review, and retest improvements.

Validation Evidence

- Every control maps to a threat or requirement and has an owner.
- Evidence identifies source revision, tool version, artifact digest, decision, and result.
- Negative and recovery paths produce the expected outcome.

Cleanup

- Revoke test identities and delete keys, tokens, vulnerable artifacts, test environments, and raw sensitive output.
- Retain only sanitized decisions, metrics, and reproducible evidence.

Interview Questions and Model Answers

1. Why is clean-room recovery necessary?

The original build environment, credentials, caches, artifacts, or control plane may be untrustworthy.

2. What should be quarantined?

Potentially affected artifacts, workers, caches, credentials, signing identities, registries or channels, and deployment references.

3. How is scope determined?

Correlate identity, time, source, builder, artifact, SBOM, provenance, registry, deployment, runtime, and audit evidence.

4. When is recovery complete?

Business service and the trust chain are restored, affected scope is addressed, evidence is retained, and improvements are verified.

Chapter Summary

- Supply-chain incidents compromise release trust.
- Containment includes identities and artifacts.
- Clean-room recovery excludes suspect inputs.
- Trust restoration is distinct from service restoration.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 55: Enterprise DevSecOps Platform, Governance and Final Security Capstone

Integrating reusable controls, developer experience, risk governance, evidence, incident response and measurable outcomes.

Why this chapter matters: The final security project proves that the organization can deliver securely at scale without forcing every team to assemble a unique, unsupported control chain.

Learning Objectives

- Explain platform product in a production delivery system.
- Design golden paths and escape hatches with explicit trust boundaries.
- Operate governance and service levels using measurable evidence.
- Troubleshoot capstone evidence and its failure modes.
- Connect control decisions to threats, ownership, exceptions, and recovery.

Core Concepts

Platform product

Provide versioned templates, runner tiers, dependency proxy, security tests, SBOM/provenance/signing services, policy bundles, evidence APIs, dashboards, documentation, support, and deprecation as a product.

Golden paths and escape hatches

Make the safest path the easiest path for common workload classes. Define proportional controls by risk and a governed exception or custom path for uncommon needs.

Governance and service levels

Assign product, platform, security, operations, risk, audit, and business ownership. Measure feedback latency, release coverage, control failure, remediation exposure, exception debt, adoption, uptime, and recovery objectives.

Capstone evidence

Produce requirements, threat model, architecture, trust boundaries, identity map, repositories, pipelines, policies, artifacts, SBOM, provenance, signatures, findings, exceptions, dashboards, incident and clean-room recovery evidence.

Architecture and Operating Decisions

- Design for multiple languages, repositories, teams, risk tiers, and deployment targets.
- Use reusable components with pinned versions and compatibility contracts.
- Automate evidence but preserve human accountability for risk decisions.
- Run user research and measure adoption and friction.
- Present residual risk, cost, operating model, and roadmap honestly.

Reference Configuration or Command Pattern

```
devsecops-platform/
  contracts/      # requirements and risk tiers
  templates/     # secure pipeline and repository defaults
  runners/       # trust-tier definitions and images
  policy/        # tested versioned rule bundles
  evidence/      # schema, index and retention
  incident/      # containment and clean-room runbooks
  docs/          # onboarding, support and deprecation
  capstone-proof/ # sanitized reproducible evidence
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Teams create shadow pipelines	Golden path is slow or incomplete	Measure friction, add supported use cases and migration help	Adoption funnel
Controls differ by repository	Templates are copied rather than versioned	Use reusable components and managed upgrades	Version coverage
Security becomes release approver for everything	Risk tiers and ownership are unclear	Automate policy and assign accountable service/risk owners	Decision latency
Capstone cannot prove recovery	Evidence shows only successful scans	Run compromise, revocation and clean rebuild	Recovery result

Hands-on Lab 55: Deliver the Enterprise DevSecOps Platform Capstone

Prerequisites

- A disposable organization, repositories, registry, CI platform, Kubernetes or cloud sandbox, and evidence store.
- Two sample services with different risk profiles.
- A cross-functional review team and synthetic incident scenario.

Implementation

1. Define two workload classes, three risk tiers, consumers, SLOs, support, compliance, evidence, and recovery requirements.
2. Create secure repository and pipeline templates with isolated execution and short-lived identity.
3. Integrate dependency governance, SAST, secrets, IaC/container, DAST/API, SBOM, provenance, signing, and policy.
4. Build an evidence index and risk/exception workflow.
5. Onboard two sample teams and measure time, failures, support, and developer feedback.
6. Run a malicious dependency or compromised runner incident and clean-room recovery.
7. Present architecture, controls, metrics, residual risk, cost, roadmap, and portfolio evidence.

Validation Evidence

- Two teams can use the platform without privileged manual intervention.
- Every release can be reconstructed from source through runtime.
- Control failure and exception paths are observable and owned.
- Clean-room recovery produces a newly trusted artifact and verified service.

Cleanup

- Destroy all sandbox environments, test registries, identities, keys, tokens, evidence containing sensitive data, and vulnerable artifacts.
- Archive only sanitized capstone diagrams, source, policies, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. What is a DevSecOps platform product?

A supported set of reusable secure delivery capabilities, interfaces, documentation, telemetry, service levels, and lifecycle management.

2. How should controls vary by risk?

Use proportional, transparent risk tiers while preserving minimum nonnegotiable trust controls.

3. What makes a security golden path effective?

It is easier than bypassing, gives fast useful feedback, covers common needs, and has support and measurable outcomes.

4. What demonstrates senior DevSecOps capability?

Connecting threats, delivery, identity, evidence, risk, operations, incident recovery, developer experience, and business trade-offs.

Chapter Summary

- The platform scales secure delivery through reusable products.
- Golden paths need governed exceptions.
- Metrics balance control quality and developer flow.
- The capstone proves both normal delivery and clean recovery.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Part E Security Command and Evidence Reference

Purpose	Reference pattern
Repository history	<code>git log --show-signature --decorate --oneline</code>
Dependency inventory	Use the ecosystem lockfile plus an SCA export tied to the source revision
Container digest	<code>docker inspect --format={{index .RepoDigests 0}} <image></code>
SBOM example	<code>syft <image-or-directory> -o cyclonedx-json > sbom.json</code>
Vulnerability example	<code>grype sbom:sbom.json -o json > findings.json</code>
Secret scan example	<code>gitleaks detect --source . --report-format json --report-path secrets.json</code>
IaC scan example	Use a pinned scanner and retain rule bundle plus resolved tool version
Cosign verify	<code>cosign verify <registry>/<image>@sha256:<digest></code>
Attestation verify	<code>cosign verify-attestation --type <predicate> <image>@sha256:<digest></code>
Kubernetes dry run	<code>kubectl apply --server-side --dry-run=server -f <manifest></code>
Policy test	Run unit tests against positive, negative, exception, and outage fixtures
Evidence digest	<code>sha256sum <evidence-file></code>

Production Security Graduation Checklist

- ☐ Threat models map high-priority abuse cases to owned, testable controls.
- ☐ Repository identity, protected paths, release authority, and emergency change are governed.
- ☐ Dependencies are locked, sourced from approved origins, inventoried, and monitored.
- ☐ Untrusted builds are isolated from signing, promotion, and privileged deployment.
- ☐ Artifacts have immutable digests, SBOMs, provenance, signatures, and retention.
- ☐ SAST, SCA, secrets, IaC, container, DAST, API, and fuzzing results have contextual triage.
- ☐ Policy rules are versioned, tested, observable, and have explicit outage and exception behavior.
- ☐ Vulnerability SLAs use exploitability, reachability, asset criticality, exposure, and compensating controls.
- ☐ Evidence can reconstruct a release and supports audit without exposing sensitive data.
- ☐ Compromise containment, revocation, artifact quarantine, clean-room rebuild, and reintroduction are rehearsed.

Ten Additional DevSecOps Portfolio Projects

Project	Primary evidence	Difficulty
1. Threat-driven delivery contract	threat model, requirement-control map and ownership	Medium
2. Repository security baseline	branch rules, CODEOWNERS, secret response and access review	Medium

Project	Primary evidence	Difficulty
3. Dependency governance service	lockfiles, SCA, license policy and remediation flow	Advanced
4. Isolated runner platform	ephemeral workers, identity and egress controls	Advanced
5. Security testing orchestration	deduplication, risk normalization and developer feedback	Advanced
6. SBOM and VEX pipeline	component identity, findings and status evidence	Advanced
7. SLSA provenance project	build definition, attestation and verification	Advanced
8. Sigstore promotion gate	keyless signing, trust policy and revocation drill	Advanced
9. Kubernetes admission platform	policy tests, exceptions, outage modes and telemetry	Advanced
10. Supply-chain incident game day	containment, clean rebuild and stakeholder package	Advanced

90-Day DevSecOps Practice Plan

Weeks	Focus	Required output
1-2	Threat modeling and repository security	delivery contract, ownership map and secret drill
3-4	Dependencies and secure builds	SCA policy, isolated runner and identity evidence
5-6	Security testing and configuration	SAST/DAST/API/IaC/container test orchestration
7-8	SBOM, VEX, SLSA and signing	attestations, verification and promotion gate
9-10	Policy, risk and evidence	admission policy, exceptions and governance dashboard
11	Incident response	compromised runner and malicious artifact game day
12-13	Final DevSecOps platform	integrated capstone and technical defense

Glossary

Term	Definition
Attestation	Signed or attributable statement about an artifact, process, or property.
Build provenance	Evidence describing how, where, by whom, and from which inputs an artifact was built.

Term	Definition
CycloneDX	Specification for SBOM and related software supply-chain data.
Exploitability	Practical likelihood that a weakness can be used in the relevant environment.
in-toto	Framework for describing and verifying supply-chain steps and attestations.
Reachability	Whether vulnerable code can be invoked through the built application and deployment context.
SBOM	Software bill of materials listing components and relationships.
Sigstore	Ecosystem for signing and verifying software artifacts using transparency and identity.
SLSA	Framework for improving software artifact integrity through progressive supply-chain levels.
SPDX	Open standard for software package and license information.
VEX	Statement conveying whether a product is affected by a known vulnerability.
Waiver	Scoped, approved, evidence-backed, expiring exception to a control or remediation requirement.

Official Documentation Index

NIST Secure Software Development Framework: <https://csrc.nist.gov/Projects/ssdf>

SLSA Specification: <https://slsa.dev/spec/>

in-toto: <https://in-toto.io/>

Sigstore: <https://docs.sigstore.dev/>

Cosign: <https://docs.sigstore.dev/cosign/>

SPDX: <https://spdx.dev/>

CycloneDX: <https://cyclonedx.org/>

OWASP SAMM: <https://owasp.samm.org/>

OWASP ASVS: <https://owasp.org/www-project-application-security-verification-standard/>

OpenSSF: <https://openssf.org/>

Kubernetes Security: <https://kubernetes.io/docs/concepts/security/>

GitHub Artifact Attestations: <https://docs.github.com/actions/security-guides/using-artifact-attestations-to-establish-provenance-for-builds>

Verification note: Products, versions, certification blueprints, pricing, quotas, and feature status change. Check the current official documentation before production use or exam registration.

Volume 10 Completion

With Part E complete, Volume 10 provides an end-to-end security operating model that preserves delivery speed while making source, build, artifact, policy, deployment, risk, evidence, and recovery decisions explicit and reproducible.

Final action: Complete the labs, retain sanitized evidence, score at least 80% on the knowledge checks, and rehearse the recovery procedures before marking this part complete.